

1. Arquitetura de Dados (Structs e Vetores)

O sistema baseia-se em quatro estruturas fundamentais (`aluno`, `curso`, `turma` e `matricula`) que atuam de forma análoga às tabelas de um banco de dados relacional. Para o armazenamento em tempo de execução, foram instanciados vetores globais com capacidade máxima controlada por uma constante (`max = 50`). Cada estrutura possui atributos específicos, com destaque para as chaves primárias (PK) exclusivas para identificação individual e as chaves estrangeiras (FK) presentes na entidade associativa de matrícula, responsáveis por consolidar o relacionamento entre os dados.

2. Sistema de Validação de Entradas

Para garantir a consistência das informações em memória, o código implementa rotinas de segurança obrigatórias durante as operações de inserção (Create) e atualização (Update). A função `campoVazio` bloqueia a gravação de strings nulas oriundas de envios acidentais. O algoritmo `validarCPF` assegura o cumprimento do padrão de 14 dígitos com a devida pontuação gráfica, enquanto `validarData` checa o formato de calendário (DD/MM/AAAA). Essas barreiras evitam a corrupção do banco por erros de tipagem.

3. Mecanismo de Busca Linear

A recuperação de informações é conduzida por algoritmos de busca sequencial. Sub-rotinas como `buscarAluno` e `buscarCurso` iteram sobre os vetores ativos comparando os parâmetros recebidos com as chaves primárias alocadas — utilizando tanto a equivalência matemática direta para inteiros quanto a função `strcmp` para strings. Ao localizar a chave correspondente, o sistema devolve o índice exato daquele registro na matriz; caso contrário, retorna o flag `-1` para sinalizar a ausência do dado, guiando a lógica das operações seguintes.

4. Controle de Integridade Referencial

Um dos pilares estruturais do projeto é a prevenção contra a quebra de relacionamentos, gerenciada primariamente pela função `temMatriculaVinculada`. Antes de permitir a deleção (Delete) de um aluno, curso ou turma, o sistema executa uma varredura de segurança no vetor de matrículas. Se for detectado qualquer registro ativo atrelado à chave em processo de exclusão, a operação é imediatamente abortada. Essa lógica emula nativamente as restrições de deleção (constraints) presentes em sistemas gerenciadores de banco de dados SQL.

5. Navegação, Ordenação e Tratamento de Fluxo

A interface no terminal foi projetada com menus modulares sustentados por laços `do-while`. Um detalhe técnico essencial é a implementação sistemática da função `limparBuffer`, que higieniza o fluxo de entrada padrão (`stdin`), prevenindo saltos de leitura indesejados após o uso de `scanf`. Além disso, o sistema conta com um algoritmo

de *Bubble Sort* bidimensional que realiza permutações de registros na memória para ordenar a visualização dos alunos em ordem alfabética. A emissão de relatórios também apresenta segmentação dinâmica, permitindo recortes precisos do banco de dados.

6. Relacionamento Avançado e Consulta Cruzada

Na etapa de exibição (Read) do gerenciador de matrículas, a aplicação transcende a simples impressão de identificadores numéricos ou textuais brutos. A função `exibirMatricula` dispara gatilhos de busca automática nas tabelas base utilizando as chaves estrangeiras. Por meio dessa engenharia, o software "traduz" os IDs de forma dinâmica em tempo de execução, entregando ao usuário uma resposta limpa que exibe os nomes literais do aluno, do curso e da turma associados ao registro.

link da apresentação no youtube:  Sistema de Gestão Acadêmica em C

link do repositório no github: [Repositório - Gestão Acadêmica](#)